Lab5 Ridge &Lasso

Jupyter Notebook — original cells shown separately and in order

CELL 01 | CODE | EXECUTION COUNT: 27

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

plt.style.use("ggplot")
```

CELL 02 | CODE | EXECUTION COUNT: 28

```
df = pd.read_csv("data 5.csv")

df.head()
```

OUTPUT

```
         id diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0    842302         M        17.99         10.38          122.80     1001.0
1    842517         M        20.57         17.77          132.90     1326.0
2  84300903         M        19.69         21.25          130.00     1203.0
3  84348301         M        11.42         20.38           77.58      386.1
4  84358402         M        20.29         14.34          135.10     1297.0

   smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
0          0.11840           0.27760          0.3001              0.14710
1          0.08474           0.07864          0.0869              0.07017
2          0.10960           0.15990          0.1974              0.12790
3          0.14250           0.28390          0.2414              0.10520
4          0.10030           0.13280          0.1980              0.10430

   ...  texture_worst  perimeter_worst  area_worst  smoothness_worst  \
0  ...          17.33           184.60      2019.0            0.1622
1  ...          23.41           158.80      1956.0            0.1238
2  ...          25.53           152.50      1709.0            0.1444
3  ...          26.50            98.87       567.7            0.2098
4  ...          16.67           152.20      1575.0            0.1374

   compactness_worst  concavity_worst  concave points_worst  symmetry_worst  \
0             0.6656           0.7119                0.2654          0.4601
1             0.1866           0.2416                0.1860          0.2750
2             0.4245           0.4504                0.2430          0.3613
3             0.8663           0.6869                0.2575          0.6638
4             0.2050           0.4000                0.1625          0.2364

   fractal_dimension_worst  Unnamed: 32
0                  0.11890          NaN
1                  0.08902          NaN
2                  0.08758          NaN
3                  0.17300          NaN
4                  0.07678          NaN

[5 rows x 33 columns]
```

CELL 03 | CODE | EXECUTION COUNT: 29

```
print("Shape:", df.shape)

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDataset Information:")
df.info()

df.describe()
```

OUTPUT

```
Shape: (569, 33)

Missing Values:
id                         0
diagnosis                  0
radius_mean                0
texture_mean               0
perimeter_mean             0
area_mean                  0
smoothness_mean            0
compactness_mean           0
concavity_mean             0
concave points_mean        0
symmetry_mean              0
fractal_dimension_mean     0
radius_se                  0
texture_se                 0
perimeter_se               0
area_se                    0
smoothness_se              0
compactness_se             0
concavity_se               0
concave points_se          0
symmetry_se                0
fractal_dimension_se       0
radius_worst               0
texture_worst              0
perimeter_worst            0
area_worst                 0
smoothness_worst           0
compactness_worst          0
concavity_worst            0
concave points_worst       0
symmetry_worst             0
fractal_dimension_worst    0
Unnamed: 32              569
dtype: int64

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   id                       569 non-null    int64
 1   diagnosis                569 non-null    object
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
 32  Unnamed: 32              0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB

                 id  radius_mean  texture_mean  perimeter_mean    area_mean  \
count  5.690000e+02   569.000000    569.000000      569.000000   569.000000
mean   3.037183e+07    14.127292     19.289649       91.969033   654.889104
std    1.250206e+08     3.524049      4.301036       24.298981   351.914129
min    8.670000e+03     6.981000      9.710000       43.790000   143.500000
25%    8.692180e+05    11.700000     16.170000       75.170000   420.300000
50%    9.060240e+05    13.370000     18.840000       86.240000   551.100000
75%    8.813129e+06    15.780000     21.800000      104.100000   782.700000
max    9.113205e+08    28.110000     39.280000      188.500000  2501.000000

       smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
```

```
count       569.000000        569.000000      569.000000           569.000000
mean          0.096360          0.104341        0.088799             0.048919
std           0.014064          0.052813        0.079720             0.038803
min           0.052630          0.019380        0.000000             0.000000
25%           0.086370          0.064920        0.029560             0.020310
50%           0.095870          0.092630        0.061540             0.033500
75%           0.105300          0.130400        0.130700             0.074000
max           0.163400          0.345400        0.426800             0.201200

       symmetry_mean  ...  texture_worst  perimeter_worst   area_worst  \
count     569.000000  ...     569.000000       569.000000   569.000000
mean        0.181162  ...      25.677223       107.261213   880.583128
std         0.027414  ...       6.146258        33.602542   569.356993
min         0.106000  ...      12.020000        50.410000   185.200000
25%         0.161900  ...      21.080000        84.110000   515.300000
50%         0.179200  ...      25.410000        97.660000   686.500000
75%         0.195700  ...      29.720000       125.400000  1084.000000
max         0.304000  ...      49.540000       251.200000  4254.000000

       smoothness_worst  compactness_worst  concavity_worst  \
count        569.000000         569.000000       569.000000
mean           0.132369           0.254265         0.272188
std            0.022832           0.157336         0.208624
min            0.071170           0.027290         0.000000
25%            0.116600           0.147200         0.114500
50%            0.131300           0.211900         0.226700
75%            0.146000           0.339100         0.382900
max            0.222600           1.058000         1.252000

       concave points_worst  symmetry_worst  fractal_dimension_worst  \
count            569.000000      569.000000               569.000000
mean               0.114606        0.290076                 0.083946
std                0.065732        0.061867                 0.018061
min                0.000000        0.156500                 0.055040
25%                0.064930        0.250400                 0.071460
50%                0.099930        0.282200                 0.080040
75%                0.161400        0.317900                 0.092080
max                0.291000        0.663800                 0.207500

       Unnamed: 32
count          0.0
mean           NaN
std            NaN
min            NaN
25%            NaN
50%            NaN
75%            NaN
max            NaN

[8 rows x 32 columns]
```

CELL 04 | CODE | EXECUTION COUNT: 30

```
# Preprocessing
# Drop unnecessary ID and completely empty Unnamed columns if they exist
df = df.drop(columns=["id", "Unnamed: 32"], errors="ignore")

# Map diagnosis target to binary (M = 1, B = 0)
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

# Features and Target
X = df.drop("diagnosis", axis=1)
y = df["diagnosis"]

# Handle missing values if any
imputer = SimpleImputer(strategy='mean')
X = imputer.fit_transform(X)

print("Shape after preprocessing:", X.shape)
```

OUTPUT

```
Shape after preprocessing: (569, 30)
```

CELL 05 | CODE | EXECUTION COUNT: 31

```
import seaborn as sns

plt.figure(figsize=(10,8))

sns.heatmap(
    df.corr(),
    cmap='coolwarm',
    annot=True
)

plt.title("Correlation Heatmap")
plt.show()
```

OUTPUT

```
<Figure size 1000x800 with 2 Axes>
```

CELL 06 | CODE | EXECUTION COUNT: 32

```
# Train-Test Split & Standardization
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

OUTPUT

```
Training samples: 455
Testing samples: 114
```

CELL 07 | CODE | EXECUTION COUNT: 33

```
# Train Ridge Model
ridge = LogisticRegression(
    penalty='l2',
    solver='liblinear',
    C=0.1,
    random_state=42
)

ridge.fit(X_train, y_train)

ridge_pred = ridge.predict(X_test)

ridge_accuracy = accuracy_score(y_test, ridge_pred)

print("Ridge Accuracy:", round(ridge_accuracy,4))
```

OUTPUT

```
Ridge Accuracy: 0.9912
```

CELL 08 | CODE | EXECUTION COUNT: 34

```
# Train Lasso Model
lasso = LogisticRegression(
    penalty='l1',
    solver='liblinear',
    C=0.05,
    random_state=42
)

lasso.fit(X_train, y_train)

lasso_pred = lasso.predict(X_test)

lasso_accuracy = accuracy_score(y_test, lasso_pred)

print("Lasso Accuracy:", round(lasso_accuracy,4))
```

OUTPUT

```
Lasso Accuracy: 0.9649
```

CELL 09 | CODE | EXECUTION COUNT: 35

```
# Compare accuracy
results = pd.DataFrame({

    "Model":["Ridge","Lasso"],

    "Accuracy":[ridge_accuracy,
                lasso_accuracy]

})

results
```

OUTPUT

```
    Model  Accuracy
0  Ridge  0.991228
1  Lasso  0.964912
```

CELL 10 | CODE | EXECUTION COUNT: 36

```
plt.figure(figsize=(6,4))

plt.bar(results["Model"],
        results["Accuracy"])

plt.title("Model Accuracy Comparison")

plt.ylabel("Accuracy")

plt.show()
```

OUTPUT

```
<Figure size 600x400 with 1 Axes>
```

CELL 11 | CODE | EXECUTION COUNT: 37

```
print("Ridge Confusion Matrix")

print(confusion_matrix(y_test,ridge_pred))

print(classification_report(y_test,ridge_pred))
```

OUTPUT

```
Ridge Confusion Matrix
[[71  0]
 [ 1 42]]
              precision    recall  f1-score   support

           0       0.99      1.00      0.99        71
           1       1.00      0.98      0.99        43

    accuracy                           0.99       114
   macro avg       0.99      0.99      0.99       114
weighted avg       0.99      0.99      0.99       114
```

CELL 12 | CODE | EXECUTION COUNT: 38

```
print("Lasso Confusion Matrix")

print(confusion_matrix(y_test,lasso_pred))

print(classification_report(y_test,lasso_pred))
```

OUTPUT

```
Lasso Confusion Matrix
[[69  2]
 [ 2 41]]
              precision    recall  f1-score   support

           0       0.97      0.97      0.97        71
           1       0.95      0.95      0.95        43

    accuracy                           0.96       114
   macro avg       0.96      0.96      0.96       114
weighted avg       0.96      0.96      0.96       114
```

CELL 13 | CODE | EXECUTION COUNT: 39

```
# Coefficient Comparison
feature_names = df.drop("diagnosis", axis=1).columns

coef = pd.DataFrame({
    "Feature":feature_names,
    "Ridge":ridge.coef_[0],
    "Lasso":lasso.coef_[0]
})

coef.round(3)
coef
```

OUTPUT

```
                   Feature     Ridge     Lasso
0                radius_mean  0.357403  0.000000
1               texture_mean  0.398598  0.000000
2             perimeter_mean  0.347498  0.000000
3                  area_mean  0.374550  0.000000
4            smoothness_mean  0.125217  0.000000
5           compactness_mean -0.024754  0.000000
6             concavity_mean  0.378562  0.000000
7        concave points_mean  0.479329  0.868361
8              symmetry_mean  0.028115  0.000000
9     fractal_dimension_mean -0.160351  0.000000
10                radius_se  0.494552  0.000000
11               texture_se -0.029907  0.000000
12             perimeter_se  0.352548  0.000000
13                  area_se  0.394921  0.000000
14            smoothness_se  0.084735  0.000000
15           compactness_se -0.268073  0.000000
16             concavity_se -0.073278  0.000000
17        concave points_se  0.070860  0.000000
18               symmetry_se -0.146361  0.000000
19     fractal_dimension_se -0.245622  0.000000
20             radius_worst  0.497651  1.383065
21            texture_worst  0.580095  0.419819
22          perimeter_worst  0.442565  0.000000
23               area_worst  0.479280  0.000000
24         smoothness_worst  0.381606  0.000000
25        compactness_worst  0.140693  0.000000
26          concavity_worst  0.399543  0.000000
27     concave points_worst  0.447889  0.565437
28           symmetry_worst  0.468966  0.102943
29  fractal_dimension_worst  0.118588  0.000000
```

CELL 14 | CODE | EXECUTION COUNT: 40

```
# Coefficient Bar Chart
coef.set_index("Feature").plot(
    kind="bar",
    figsize=(12,6)
)

plt.title("Feature Coefficients: Ridge vs Lasso")

plt.ylabel("Coefficient Value")

plt.grid(axis="y")

plt.show()
```

OUTPUT

```
<Figure size 1200x600 with 1 Axes>
```

CELL 15 | CODE | EXECUTION COUNT: 41

```
# Count Selected Features
ridge_features = np.sum(ridge.coef_[0] != 0)
lasso_features = np.sum(lasso.coef_[0] != 0)

print("Number of Features Used")

print("Ridge :", ridge_features)

print("Lasso :", lasso_features)
```

OUTPUT

```
Number of Features Used
Ridge : 30
Lasso : 5
```